

JAVA INTERVIEW PREPARATION

CHAPTER 9

STREAMS AND COLLECTORS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Streams and Collectors: Declarative Data Processing with Explicit Contracts

Senior / Lead Java Developer Reading Chapter

The Stream API lets Java code describe a data-processing pipeline rather than manually controlling every loop, temporary collection, and branch. A stream pipeline can express selection, transformation, grouping, aggregation, and short-circuit search with a vocabulary that separates what should happen from the mechanics of iteration.

That conciseness can be misleading. Streams have precise contracts around laziness, ordering, interference, reduction, parallel execution, resource ownership, and result mutability. A senior developer should be able to explain not only what a pipeline returns, but when each stage runs, how much data it retains, whether the result is deterministic, and why a parallel version is or is not correct.

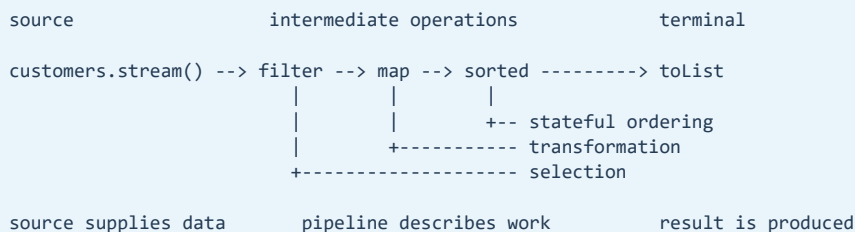
Streams are not automatically faster than loops, and pipelines are not automatically clearer. Their value comes from exposing a sequence of transformations whose semantics are easy to reason about.

The Stream Pipeline Model

A stream pipeline contains a source, zero or more intermediate operations, and one terminal operation.

```
List<String> activeCustomerNames = customers.stream()
    .filter(Customer::isActive)
    .map(Customer::displayName)
    .sorted()
    .toList();
```

The parts have different responsibilities:



Intermediate operations normally return another Stream and are lazy. The terminal operation triggers traversal and produces a value or side effect. Common terminal operations include `toList`, `collect`, `reduce`, `count`, `forEach`, `findFirst`, `anyMatch`, and `min`.

A Stream is not a collection. It does not own permanent element storage and cannot usually be traversed twice. It represents a one-use computation over a source.

```
Stream<Order> stream = orders.stream();

long count = stream.count();
// stream.findFirst(); // IllegalStateException: stream already operated upon
```

If repeated traversal is required, keep or recreate the source, or use a supplier that deliberately creates a new stream:

```
Supplier<Stream<Order>> paidOrders = () ->
    orders.stream().filter(Order::isPaid);
```

Internal Iteration and Control Ownership

An external loop gives application code direct control over traversal:

```
List<Invoice> invoices = new ArrayList<>();

for (Order order : orders) {
    if (order.isPaid()) {
        invoices.add(invoiceFactory.create(order));
    }
}
```

The equivalent pipeline delegates traversal to the Stream implementation:

```
List<Invoice> invoices = orders.stream()
    .filter(Order::isPaid)
    .map(invoiceFactory::create)
    .toList();
```

This is internal iteration. The library controls how elements move through the stages. That enables lazy fusion, short-circuiting, and possible parallel decomposition, but it also means callbacks must follow the receiving API's execution rules.

Streams work best when the operations describe independent element transformations. A loop may remain clearer when control flow involves several mutable accumulators, complex early exits, checked-exception recovery, or a state machine whose transitions are central to the algorithm.

The goal is not to eliminate loops. It is to choose the representation that makes correctness most visible.

Laziness, Fusion, and Short-Circuiting

An intermediate operation does not normally process the source when it is declared.

```
Stream<Order> pipeline = orders.stream()
    .filter(order -> {
        logger.debug("filtering {}", order.id());
        return order.isPaid();
    })
    .map(invoiceFactory::create);

// Nothing has been traversed yet.
List<Invoice> invoices = pipeline.toList();
```

Once a terminal operation begins, stateless operations such as `filter` and `map` can be fused. One element flows through multiple stages before the next element is requested.

```
Order A --> filter --> map --> terminal
Order B --> filter -----X
Order C --> filter --> map --> terminal
```

The implementation does not need to create a complete temporary List after every stage.

Short-circuit operations may stop traversal as soon as the answer is known:

```
boolean hasOverdueOrder = orders.stream()
    .filter(Order::isOpen)
    .anyMatch(order -> order.dueDate().isBefore(today));
```

`anyMatch` can stop after the first match. `allMatch`, `noneMatch`, `findFirst`, `findAny`, `limit`, and `takeWhile` can also limit traversal under their contracts.

Operation order affects cost. Put a cheap selective filter before an expensive transformation when semantics permit it:

```
orders.stream()
    .filter(Order::isPaid)
    .map(expensiveInvoiceRenderer::render)
    .limit(20)
    .toList();
```

However, do not reorder operations blindly. Moving `limit` before `filter` changes the question from "first twenty paid orders" to "paid orders among the first twenty orders."

Stateless and Stateful Intermediate Operations

Stateless operations can process an element without remembering earlier elements. Examples include `map`, `filter`, `peek`, and usually `flatMap`.

Stateful operations need information about multiple elements. Examples include `sorted`, `distinct`, `limit`, `skip`, `takeWhile`, and `dropWhile`, although the amount of state and parallel cost differs.

```
List<Customer> highestValue = customers.stream()
    .sorted(comparing(Customer::lifetimeValue).reversed())
    .limit(100)
    .toList();
```

Sorting generally needs to observe and retain the input before producing globally ordered output. On a large or unbounded stream, this can dominate memory and latency. `distinct` must track values it has encountered. Ordered `limit` in parallel may require coordination to preserve which elements count as first.

Ask whether the source can bound the work earlier. A database query with an appropriate index and `ORDER BY ... LIMIT 100` may be far better than loading millions of rows and sorting them in Java.

map and flatMap

`map` converts each input into exactly one output value, although that value can itself be a collection or `Optional`.

```
List<CustomerId> ids = customers.stream()
    .map(Customer::id)
    .toList();
```

If each `Order` contains multiple lines, `map` produces a stream of `Lists`:

```
Stream<List<OrderLine>> nested = orders.stream()
    .map(Order::lines);
```

`flatMap` converts each input into a stream and concatenates those streams into one logical stream:

```
List<ProductId> purchasedProducts = orders.stream()
    .flatMap(order -> order.lines().stream())
    .map(OrderLine::productId)
    .distinct()
    .toList();
```

```
Order 1 --> [Line A, Line B] --+
                                +--> A, B, C --> ProductIds
Order 2 --> [Line C] -----+
```

The same idea works with `Optional.stream()` to keep present values and discard empty ones:

```
List<Customer> resolved = customerIds.stream()
    .map(repository::findById)
    .flatMap(Optional::stream)
    .toList();
```

Be careful: this pipeline silently drops missing customers. If absence is a business error, collect failures explicitly or stop with a meaningful exception instead of flattening them away.

Encounter Order and Determinism

A stream may have an encounter order derived from its source. A List stream follows list order. A `TreeSet` follows sorted order. A `HashSet` does not promise a stable encounter order.

Operations such as `forEachOrdered`, `findFirst`, `limit`, and `skip` can depend on encounter order. `findAny` permits any element and gives parallel execution more freedom.

```
Optional<Order> first = orders.stream()
    .filter(Order::requiresReview)
    .findFirst();
```

This means the first matching Order according to source encounter order. If the source is an unordered Set, "first" has no domain meaning.

Calling `unordered()` does not shuffle elements. It removes the requirement that later operations preserve encounter order, which can enable cheaper parallel execution when order is irrelevant.

Never rely on the accidental iteration order of an unordered source. If an API promises order, create it explicitly with a sorted source, an order-preserving collection, or `sorted` with a complete comparator. A comparator should include a deterministic tie-breaker when equal primary values must still produce stable business output.

Side Effects and Non-Interference

Stream functions should normally avoid modifying the source or shared external state. The non-interference rule means the source should not be structurally changed while the pipeline is executing unless the source explicitly supports that behavior.

This code is unsafe and conceptually wrong:

```
List<Order> paid = new ArrayList<>();

orders.parallelStream()
    .filter(Order::isPaid)
    .forEach(paid::add); // ArrayList mutation races
```

Use a collector:

```
List<Order> paid = orders.parallelStream()
    .filter(Order::isPaid)
    .toList();
```

Even a synchronized List would turn the mutation into contention and hide the pipeline's result behind a side effect. Collection is designed to combine partial results correctly.

Side effects are valid when they are the actual terminal purpose, such as publishing events, but the contract must remain explicit:

```
approvedInvoices.stream().forEach(invoicePublisher::publish);
```

Ask whether ordering, retries, partial failure, idempotency, and transaction boundaries are defined. A stream does not turn a sequence of remote calls into an atomic operation.

peek Is Primarily an Observation Tool

`peek` is an intermediate operation. It runs only for elements requested by the terminal operation and may not run for every source element when the pipeline short-circuits.

```
Optional<Order> firstLargeOrder = orders.stream()
    .peek(order -> logger.debug("considering {}", order.id()))
    .filter(order -> order.total().compareTo(limit) > 0)
    .findFirst();
```

Using `peek` for required mutation is fragile:

```
// Avoid hiding essential work in peek.
orders.stream()
    .peek(Order::markProcessed)
    .map(this::createReceipt)
    .toList();
```

The name suggests observation, not a required state transition. Put important transformations in `map`, explicit workflows, or well-named terminal actions. Also remember that logging every element can create severe I/O and storage volume in production.

Reduction and the Algebra Behind It

Reduction combines many elements into one result.

```
BigDecimal total = orders.stream()
    .map(Order::total)
    .reduce(BigDecimal.ZERO, BigDecimal::add);
```

The identity must be a neutral value: combining it with any element should return that element. The accumulator must be associative for parallel reduction, meaning grouping does not change the answer.

```
sequential grouping: (((a + b) + c) + d)
parallel grouping:   (a + b) + (c + d)
associative operation => same result
```

Addition is associative in exact arithmetic. Subtraction is not:

```
int result = numbers.parallelStream()
    .reduce(0, (left, right) -> left - right); // invalid parallel reduction
```

Floating-point addition is mathematically associative but not exactly associative under finite precision. Parallel grouping can therefore change the last bits. Financial aggregation should generally use `BigDecimal` with an explicit scale and rounding policy, or a domain `Money` type.

The three-argument `reduce` has identity, accumulator, and combiner:

```
int totalLength = words.parallelStream().reduce(
    0,
    (length, word) -> length + word.length(),
    Integer::sum
);
```

The accumulator combines a partial result with an element. The combiner merges partial results. Their contracts must be compatible. Mutable accumulation is normally better represented by `collect`, not by mutating an identity inside `reduce`.

Mutable Reduction with Collectors

A Collector describes how to create a result container, incorporate elements, combine partial containers, and optionally finish the result.

```
supplier      -> create partial container
accumulator   -> add one element
combiner      -> merge partial containers
finisher      -> convert accumulated form to final form
characteristics -> declare optimization-relevant properties
```

Common factories cover most requirements:

```
Map<Status, List<Order>> ordersByStatus = orders.stream()
    .collect(Collectors.groupingBy(Order::status));

Map<Boolean, List<Order>> partitioned = orders.stream()
    .collect(Collectors.partitioningBy(Order::isPaid));

Set<CustomerId> customerIds = orders.stream()
    .map(Order::customerId)
    .collect(Collectors.toSet());
```

Collectors can use downstream collectors to aggregate within each group:

```
Map<Status, BigDecimal> totalsByStatus = orders.stream()
    .collect(Collectors.groupingBy(
        Order::status,
        Collectors.mapping(
            Order::total,
            Collectors.reducing(BigDecimal.ZERO, BigDecimal::add)
        )
    ));
```

An alternative using `toMap` can be clearer:

```
Map<Status, BigDecimal> totalsByStatus = orders.stream()
    .collect(Collectors.toMap(
        Order::status,
        Order::total,
        BigDecimal::add,
        EnumMap::new
    ));
```

The merge function is essential when keys can repeat. Without it, `toMap` throws an exception on duplicate keys.

```
Map<String, Customer> byEmail = customers.stream()
    .collect(Collectors.toMap(
        customer -> normalize(customer.email()),
        Function.identity(),
        this::chooseAuthoritativeRecord
    ));
```

Do not choose `(first, second) -> first` merely to suppress the duplicate error. Decide whether duplicates indicate bad data, and define an auditable resolution policy.

Result Mutability and Collection Factories

Result mutability is part of an API contract.

`Stream.toList()` returns an unmodifiable List. It may contain null elements if the stream contains nulls.

```
List<String> names = customers.stream()
    .map(Customer::displayName)
    .toList();

// names.add("Another"); // UnsupportedOperationException
```

`Collectors.toUnmodifiableList()` also returns an unmodifiable List, but rejects null elements.

`Collectors.toList()` does not promise a particular implementation, mutability, serializability, or thread safety, even though common implementations currently return a mutable ArrayList.

If a specific mutable type is required, request it explicitly:

```
ArrayList<String> names = customers.stream()
    .map(Customer::displayName)
    .collect(Collectors.toCollection(ArrayList::new));
```

If the public API should return immutable data, consider `List.copyOf` or an unmodifiable collector and document null policy. Do not let callers depend on incidental collector implementation details.

Advanced Collector Composition

Downstream collectors let one traversal produce structured results.

```
Map<Department, Long> employeeCounts = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.counting()
    ));
```

`collectingAndThen` applies a finisher:


```
Map<Department, List<Employee>> immutableByDepartment = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.collectingAndThen(
            Collectors.toList(),
            List::copyOf
        )
    ));
```

teeing, added in Java 12, sends each element to two downstream collectors and merges their results:

```
record Range(BigDecimal minimum, BigDecimal maximum) {}

Range range = orders.stream()
    .map(Order::total)
    .collect(Collectors.teeing(
        Collectors.minBy(BigDecimal::compareTo),
        Collectors.maxBy(BigDecimal::compareTo),
        (minimum, maximum) -> new Range(
            minimum.orElse(BigDecimal.ZERO),
            maximum.orElse(BigDecimal.ZERO)
        )
    ));
```

Advanced composition is useful when it remains understandable. A deeply nested collector can be harder to maintain than a named accumulator type with explicit methods.

Designing a Custom Collector

Create a custom Collector only when existing collectors cannot express the operation clearly. Suppose a report needs a count and exact total:

```
final class OrderStatistics {
    private long count;
    private BigDecimal total = BigDecimal.ZERO;

    void add(Order order) {
        count++;
        total = total.add(order.total());
    }

    OrderStatistics merge(OrderStatistics other) {
        count += other.count;
        total = total.add(other.total);
        return this;
    }

    OrderSummary finish() {
        return new OrderSummary(count, total);
    }
}
```

```
Collector<Order, OrderStatistics, OrderSummary> summaryCollector =
    Collector.of(
        OrderStatistics::new,
        OrderStatistics::add,
        OrderStatistics::merge,
        OrderStatistics::finish
    );
```

Each parallel partition receives its own accumulator unless the collector correctly declares concurrent accumulation. The combiner must merge without losing information, and the finisher must not expose mutable internal state accidentally.

Collector characteristics are promises, not performance hints to add casually:

- `IDENTITY_FINISH` means the accumulator type can be returned as the result without transformation.
- `UNORDERED` means the result does not depend on encounter order.
- `CONCURRENT` means the accumulator supports concurrent calls under the collector contract.

An incorrect characteristic can create data corruption under parallel execution.

Parallel Streams: Correctness Before Speed

Calling `parallelStream()` or `parallel()` allows the pipeline to divide work, usually through the common `ForkJoinPool`.

```
source Splitterator
|
+---- partition A ----> partial result A --+
+---- partition B ----> partial result B ----+--> combine
+---- partition C ----> partial result C --+
```

Parallel execution is most promising when:

- The input is large enough to amortize splitting and coordination.
- Work per element is CPU-intensive and independent.
- The source splits efficiently, as arrays and ArrayLists do.
- Reduction is associative and has a correct identity.
- Ordering is unnecessary or inexpensive.
- The environment has available CPU capacity.

It is often unsuitable when:

- The data set is small.
- Operations block on databases, HTTP, files, or locks.
- The pipeline mutates shared state.
- Strict encounter order requires coordination.
- The application server shares the common pool with unrelated work.
- Latency depends on thread-local transaction, tracing, or security context.

Parallel streams do not let application code directly express executor ownership in the pipeline API. Running a parallel stream from a custom `ForkJoinPool` has subtle implementation and maintenance implications and should not be treated as a general executor-selection mechanism. For explicit asynchronous workflows, use an executor-based API whose ownership, queueing, timeouts, and observability are deliberate.

Always benchmark representative workloads in the real deployment environment. A laptop microbenchmark does not capture container CPU quotas, concurrent traffic, pool contention, garbage collection, or downstream limits.

Spliterators and Source Quality

A `Spliterator` is the traversal and partitioning abstraction behind streams. It can report characteristics such as `ORDERED`, `SIZED`, `SORTED`, `DISTINCT`, `NONNULL`, `IMMUTABLE`, and `CONCURRENT`.

Good parallel performance depends on balanced splitting and accurate size information. An `ArrayList` can split a contiguous index range efficiently. A linked structure, iterator-backed source, or expensive generator may split poorly.

Custom Spliterators are advanced infrastructure. Their contracts cover traversal, splitting, estimated size, comparator behavior, and characteristics. Incorrect claims can produce missed elements, duplicates, ordering violations, or broken optimizations. Prefer library-provided sources unless the domain truly requires a custom traversal abstraction.

Primitive Streams and Boxing

`IntStream`, `LongStream`, and `DoubleStream` provide primitive operations and specialized reductions.

```
int totalQuantity = orders.stream()
    .flatMap(order -> order.lines().stream())
    .mapToInt(OrderLine::quantity)
    .sum();
```

They avoid boxing every primitive into a wrapper and provide operations such as `sum`, `average`, and `summaryStatistics`.

```
IntSummaryStatistics statistics = responseTimes.stream()
    .mapToInt(Duration::toMillisPart)
    .summaryStatistics();
```

Use the correct duration conversion for the domain; `toMillisPart` is only the millisecond component, while `toMillis` is the full duration. This distinction illustrates a broader senior-level point: concise pipelines still require precise API semantics.

Boxing overhead matters in high-volume numeric work, but primitive streams cannot represent every domain type and may reduce readability when conversions multiply. Profile allocation and CPU before optimizing routine business pipelines.

Nulls, Optional Results, and Failure Semantics

Streams can technically contain nulls, but many operations and collectors handle them inconsistently. Null also makes method references such as `String::trim` fail.

```
List<String> normalized = values.stream()
    .filter(Objects::nonNull)
    .map(String::trim)
    .filter(value -> !value.isEmpty())
    .toList();
```

Filtering null is correct only when dropping it is the domain policy. Otherwise validate and fail with context.

Search operations return `Optional` because a result may not exist:

```
Order largest = orders.stream()
    .max(comparing(Order::total))
    .orElseThrow(() -> new NoOrdersFoundException(customerId));
```

Exceptions thrown inside a pipeline propagate through the terminal operation. Standard functional interfaces do not declare checked exceptions, so checked failures require handling or translation at an appropriate boundary. Do not convert every failure into an empty `Optional`; absence and operational failure are different states.

Streams over I/O Resources

Most collection streams do not require closing. Some streams backed by I/O resources do, including those returned by methods such as `Files.lines` and `Files.list`.

```
try (Stream<String> lines = Files.lines(path, StandardCharsets.UTF_8)) {
    long errorCount = lines
        .filter(line -> line.contains("ERROR"))
        .count();
}
```

The terminal operation does not automatically close every resource-backed stream. Use try-with-resources and keep consumption inside the resource lifetime.

Do not return a lazy Stream whose underlying file, database cursor, or network resource has already been closed:

```
// Dangerous ownership: who closes the resource and when?
Stream<String> readLines(Path path) throws IOException {
    return Files.lines(path);
}
```

An API returning such a stream must document that the caller owns closing it, but returning a fully materialized result or accepting a callback may provide safer ownership. Database frameworks also attach streams to transactions and persistence contexts; consuming them after the boundary closes can fail or leak resources.

Performance and the Database Boundary

A visually elegant pipeline can still perform disastrous work:

```
List<CustomerView> views = customers.stream()
    .map(customer -> new CustomerView(
        customer,
        orderRepository.findByCustomerId(customer.id())
    ))
    .toList();
```

This may issue one database query per customer, producing an N+1 problem. The Stream API neither causes nor prevents it; concise syntax simply makes the repeated call easier to overlook.

Push filtering, joining, aggregation, sorting, and pagination to the database when it can execute them efficiently with indexes and set-based operations. Bring data into Java when business logic, external service composition, or in-memory algorithms justify it.

For large pipelines, inspect:

- Source cardinality and whether it is bounded.
- Allocation rate from mapping, boxing, and collection.
- Retained state from sorting, distinct, and grouping.
- Database and network calls inside callbacks.
- Short-circuit opportunities.
- Result size and whether streaming actually avoids materialization.
- Parallel-pool saturation and CPU quotas.

Measure the entire workflow, not only the pipeline expression.

Readability, Debugging, and Testing

A pipeline should read like a sequence of named transformations:

```
List<PaymentInstruction> instructions = orders.stream()
    .filter(this::isEligibleForPayment)
    .map(this::createPaymentInstruction)
    .sorted(bySettlementPriority())
    .toList();
```

Extract long lambdas and nested collector configuration. Named methods provide test seams, meaningful stack-trace frames, and a place for domain documentation.

Test more than the happy result:

- Empty input.
- One element.
- Duplicate keys and merge behavior.
- Null policy.
- Comparator ties.
- Boundary values for `limit`, `skip`, and matching.
- Sequential and parallel results when parallel use is supported.
- Exception propagation and resource closing.
- Result mutability guarantees.

Property-based tests are useful for reduction laws. For an intended parallel reduction, test that different input partitions and encounter orders produce the same valid result. A few example cases may not expose a non-associative accumulator.

For debugging, temporarily extract stages or collect a small bounded sample. Use `peek` carefully for observation, never as a permanent high-volume logging system. Production telemetry should be placed at meaningful workflow boundaries with counts, duration, failure categories, and cardinality rather than one log per element.

Production Perspective

Stream incidents usually result from hidden cost or an incomplete execution contract. A service loads an entire table before filtering. A `groupingBy` retains millions of lists and exhausts the heap. A `toMap` deployment begins failing when real data contains duplicate normalized keys. A parallel stream blocks the common pool on HTTP calls and slows unrelated work. A callback mutates a shared object and returns nondeterministic results. A file-backed stream is never closed. A pipeline silently removes missing entities through `Optional.stream`. An unmodifiable `toList` result reaches legacy code that expects mutation.

When investigating a stream pipeline, reconstruct the execution model:

- What is the source, its size, order, ownership, and splitting behavior?
- Which operations are lazy, stateful, short-circuiting, or side-effecting?
- What terminal operation triggers traversal?
- Which stages retain all or part of the input?
- Are callbacks performing I/O, locking, mutation, or nested queries?
- Does reduction obey identity, associativity, and combiner compatibility?
- What mutability and null contract does the result provide?
- Is the stream resource-backed and correctly closed?
- If parallel, which pool is used and what else depends on it?

This turns a compact pipeline back into an explicit operational model. Only then should optimization or refactoring begin.

Interview-Ready Summary

A Stream is a one-use computation over a source, not a data structure. A pipeline consists of a source, lazy intermediate operations, and a terminal operation that triggers traversal. Internal iteration lets the library fuse stages, short-circuit work, and potentially partition the source.

Stateless operations such as filter and map can process elements independently. Stateful operations such as sorted and distinct retain cross-element information and can be expensive for large or parallel sources. Operation order affects both semantics and cost.

`map` produces one output value per input; `flatMap` produces a stream per input and flattens the nested results. Encounter order comes from the source and affects operations such as `findFirst`, ordered `limit`, and `forEachOrdered`. Unordered sources have no meaningful first element unless order is imposed explicitly.

Stream behavior should normally be non-interfering and avoid shared mutable state. Collectors perform mutable reduction through suppliers, accumulators, combiners, and finishers. Duplicate-key policy, downstream aggregation, result mutability, null handling, and collector characteristics are part of correctness.

Reduction requires a neutral identity and associative operation for parallel execution. The accumulator and combiner must be compatible. Parallel streams help only for suitable large, CPU-bound, independently processed data with efficient splitting and available cores. They are a poor default for blocking I/O, shared mutation, strict ordering, and environments where common-pool ownership is unclear.

Some I/O-backed streams must be closed. Stream syntax does not remove database boundaries, transaction lifetimes, N+1 queries, memory retention, or overload concerns. Senior-level reasoning makes source cardinality, lifecycle, execution cost, and failure behavior explicit before choosing a pipeline.

Revision Notes

- A Stream is a one-use computation, not a collection that stores elements.
- A pipeline has a source, intermediate operations, and one terminal operation.
- Intermediate operations are normally lazy; the terminal operation triggers traversal.
- Stateless stages can be fused so one element flows through several operations.
- Short-circuiting may stop traversal early, but operation order changes semantics.
- Streams use internal iteration; the library owns traversal.
- Use loops when complex control flow is clearer than a pipeline.
- Stateful operations such as sorted and distinct retain cross-element information.
- Push large sorting, filtering, joining, and pagination to the database when appropriate.
- `map` creates one result per element; `flatMap` flattens nested streams.
- Do not silently discard missing domain data with `Optional.stream` unless absence is expected.
- Encounter order comes from the source; `HashSet` does not promise stable order.
- `unordered()` removes an ordering constraint; it does not shuffle elements.
- Avoid source interference and shared mutable state inside callbacks.
- Use collectors instead of mutating an external List from `forEach`.
- `peek` is for observation and may not run for every source element.
- Reduction identities must be neutral and operations associative for parallel correctness.

- Subtraction is not a valid associative parallel reduction.
- Use `collect` rather than mutating a container inside `reduce`.
- A Collector defines supplier, accumulator, combiner, finisher, and characteristics.
- `toMap` needs an intentional merge policy when duplicate keys are possible.
- `Stream.toList()` returns an unmodifiable List.
- `Collectors.toList()` does not guarantee a specific implementation or mutability.
- Use `toCollection` when a particular mutable collection type is required.
- Collector characteristics are correctness promises, not casual optimization hints.
- Parallel streams usually use the common ForkJoinPool.
- Parallelism suits large, CPU-bound, independent work with efficient splitting.
- Avoid parallel streams for blocking I/O, shared state, unclear pool ownership, or strict ordering.
- Benchmark with representative traffic, CPU quotas, contention, and downstream limits.
- Primitive streams reduce boxing and provide specialized aggregate operations.
- Define a deliberate null policy rather than assuming every stream is null-safe.
- Do not convert operational failures into empty Optional values.
- Close streams backed by files or other I/O resources with try-with-resources.
- Do not return a lazy resource-backed stream without a clear closing contract.
- Watch for N+1 queries hidden inside map, filter, or other callbacks.
- Measure source size, allocations, retained state, I/O, and result cardinality.
- Extract complex lambdas and collectors into named, independently testable components.
- Test empty input, duplicates, ordering, mutability, exceptions, and reduction laws.
- Diagnose pipelines by reconstructing source, stages, terminal trigger, retained state, and execution context.